

BASICS OF COMPUTER ORGANIZATION

AR107 - Unit I: Digital Logic Fundamentals

EXTENDED IN-DEPTH EDITION

Detailed Notes for First-Year Minor Examinations

GGSIPIU - University School of Automation & Robotics (USAR)

Syllabus coverage

Official Unit I (Digital Logic Fundamentals, 8 lectures): Number Systems & Codes - binary, octal, decimal, hexadecimal, signed numbers using 1's and 2's complement, BCD, ASCII and Gray code; Boolean Algebra and Logic Gates - postulates and theorems, truth tables, basic and universal gates, SOP/POS, K-maps for 2 to 4 variables; Combinational Logic Design - half/full adders, subtractors, multiplexers, demultiplexers, encoders, decoders and comparators.

How to use these notes

First understand each rule, then reproduce the truth table or conversion method without looking. For K-maps and combinational circuits, solve at least 3-5 problems by hand. In a Minor exam, showing steps usually matters as much as the final answer.

Prepared as syllabus-aligned study notes. Use your class instructions for the exact Minor-exam pattern and weightage.

1. Unit I at a Glance

Unit I develops the digital-logic foundation needed to understand how a CPU represents data and how basic hardware blocks are built. The unit can be studied in three connected parts: representations of data, Boolean simplification, and combinational-circuit design.

Syllabus-Aligned Checklist

- Number systems: binary, octal, decimal and hexadecimal.
- Signed numbers: 1's complement and 2's complement.
- Codes: BCD, ASCII and Gray code.
- Boolean algebra: postulates, theorems and truth tables.
- Logic gates: basic gates and universal NAND/NOR gates.
- SOP and POS forms; minterms and maxterms.
- Karnaugh maps for 2, 3 and 4 variables.
- Combinational design: adders, subtractors, multiplexers, demultiplexers, encoders, decoders and comparators.

Minor-Exam Study Priority

Priority	Topics	What you must be able to do
Very High	Number conversions; complements	Convert correctly, show steps, state n-bit range and identify overflow.
Very High	Boolean laws; K-maps	Simplify expressions and move between truth table, minterms/maxterms and K-map.
Very High	Adders/subtractors; MUX/decoder	Write truth table, derive expressions and explain circuit operation.
High	BCD, ASCII, Gray code	Encode/decode and remember key properties.
High	DEMUX, encoder, comparator	Explain purpose, truth table and applications.

Core idea

A digital system uses discrete logic levels. Binary digits become Boolean variables; Boolean expressions become gates; gates are connected to form combinational building blocks.

2. Number Systems and Positional Representation

A number system is defined by its radix (base). In a positional number system, the value of a digit depends on both the digit and its position.

$$(d_n \dots d_1 d_0 . d_{-1} d_{-2})_r = \sum(d_i * r^i)$$

System	Base (r)	Allowed digits	Example
Binary	2	0, 1	101101.01 ₂
Octal	8	0-7	735 ₈
Decimal	10	0-9	259 ₁₀
Hexadecimal	16	0-9, A-F	3A7 ₁₆

Hex digit values

A=10, B=11, C=12, D=13, E=14, F=15. One hexadecimal digit corresponds to exactly four binary bits.

2.1 Converting Any Base to Decimal

Multiply each digit by the corresponding power of the base and add the results. Positions to the left of the radix point use non-negative powers; positions to the right use negative powers.

Solved Example 2.1

Question: Convert (101101.011)₂ to decimal.

Integer part = $1*2^5 + 0*2^4 + 1*2^3 + 1*2^2 + 0*2^1 + 1*2^0 = 32+8+4+1 = 45$.

Fractional part = $0*2^{-1} + 1*2^{-2} + 1*2^{-3} = 1/4 + 1/8 = 0.375$.

Therefore, (101101.011)₂ = (45.375)₁₀.

Solved Example 2.2

Question: Convert (3A7)₁₆ to decimal.

$3*16^2 + A*16^1 + 7*16^0 = 3*256 + 10*16 + 7 = 768+160+7 = 935$.

Therefore, (3A7)₁₆ = (935)₁₀.

2.2 Decimal Integer to Another Base: Repeated Division

Repeatedly divide the decimal integer by the target base. Record every remainder. Read the remainders from bottom to top.

Solved Example 2.3

Question: Convert (45)₁₀ to binary, octal and hexadecimal.

Binary: $45/2$ gives remainders 1,0,1,1,0,1 (read upward) $\Rightarrow 101101_2$.

Octal: $45/8 = 5$ remainder 5; $5/8 = 0$ remainder 5 $\Rightarrow 55_8$.

Hex: $45/16 = 2$ remainder 13(D) $\Rightarrow 2D_{16}$.

2.3 Decimal Fraction to Another Base: Repeated Multiplication

Repeatedly multiply the fractional part by the target base. Each integer part produced becomes the next digit after the radix point. Stop when the fraction becomes zero or when the desired precision is reached.

Solved Example 2.4

Question: Convert (0.625)₁₀ to binary.

$0.625*2 = 1.250 \rightarrow$ first bit 1, remaining fraction 0.250.

$0.250*2 = 0.500 \rightarrow$ next bit 0, remaining fraction 0.500.

$0.500*2 = 1.000 \rightarrow$ next bit 1, remaining fraction 0.

So, (0.625)₁₀ = (0.101)₂.

2.4 Fast Binary-Octal-Hex Conversion

Binary to octal: group bits in sets of 3 from the radix point outward. Binary to hexadecimal: group in sets of 4. Add leading or trailing zeros only to complete a group; they do not change the value.

Binary	Octal	Binary	Hex
000	0	0000	0
001	1	0001	1
010	2	0010	2
011	3	0011	3
100	4	0100	4
101	5	0101	5
110	6	0110	6

Binary	Octal	Binary	Hex
111	7	0111	7
		1000	8
		1001	9
		1010	A
		1011	B
		1100	C
		1101	D
		1110	E
		1111	F

Solved Example 2.5

Question: Convert (1101011110.1011)₂ to octal and hexadecimal.

Octal grouping: 001 101 011 110 . 101 100 $\Rightarrow$ 1 5 3 6 . 5 4 = (1536.54)₈.

Hex grouping: 0011 0101 1110 . 1011 $\Rightarrow$ 3 5 E . B = (35E.B)₁₆.

Exam check

Always write the base as a subscript or in brackets. A correct digit string without a base can be ambiguous.

2.5 Deeper Understanding of Positional Numbers

The radix point separates positive-power positions from negative-power positions. For an n-digit integer in base r, the leftmost digit carries the largest positional weight. A digit is legal only when it is smaller than the radix; therefore 10201 is valid in base 3, but 10301 is not because digit 3 is not allowed in base 3.

Position	...	3	2	1	0	-1	-2	...
Weight in base r	...	r^3	r^2	r^1	r^0	r^{-1}	r^{-2}	...

Important vocabulary

MSD/MSB = most significant digit/bit; LSD/LSB = least significant digit/bit. The significance of a bit is its positional weight, not simply whether it is 0 or 1.

Solved Example 2.6

Question: Convert (572.34)₈ to decimal.

Use powers of 8 on both sides of the radix point.

$$5 \cdot 8^2 + 7 \cdot 8^1 + 2 \cdot 8^0 + 3 \cdot 8^{-1} + 4 \cdot 8^{-2}$$

$$= 320 + 56 + 2 + 0.375 + 0.0625 = 378.4375.$$

Therefore, $(572.34)_8 = (378.4375)_{10}$.

2.6 Fractional Conversions, Precision and Truncation

An integer conversion by repeated division must terminate because the quotient eventually becomes zero. A fractional conversion may terminate or may repeat indefinitely. For example, 0.1_{10} does not have a finite binary representation. In such cases the question normally specifies a required number of fractional bits; keep the requested precision and state that the result is approximate.

Solved Example 2.7**Question: Convert (0.1)₁₀ to binary up to 6 fractional bits.**

0.1*2=0.2 -> 0; 0.2*2=0.4 -> 0; 0.4*2=0.8 -> 0.

0.8*2=1.6 -> 1; 0.6*2=1.2 -> 1; 0.2*2=0.4 -> 0.

Thus (0.1)₁₀ is approximately (0.000110)₂ to 6 fractional bits; the pattern continues rather than terminating.**2.7 Direct Conversion Strategy for Minor Exams**

Given	Required	Fastest method
Binary / octal / hex	Decimal	Weighted positional expansion.
Decimal integer	Any base	Repeated division by target base.
Decimal fraction	Any base	Repeated multiplication by target base.
Binary	Octal	Groups of 3 bits.
Binary	Hex	Groups of 4 bits.
Octal <-> Hex	Other compact base	Go through binary; regroup 3-bit and 4-bit blocks.

Solved Example 2.8**Question: Convert (7B.9)₁₆ to octal without using decimal.**Hex to binary: 7=0111, B=1011, 9=1001 -> 01111011.1001₂.

Regroup in 3s: 001 111 011 . 100 100 -> 1 7 3 . 4 4.

Therefore (7B.9)₁₆ = (173.44)₈.**Self-check**

When converting the integer part, estimate the magnitude. Example: 3A7₁₆ is a little above 3*256=768, so a decimal result near 900 is reasonable. Magnitude checks catch many arithmetic slips.

3. Signed Binary Numbers, 1's Complement and 2's Complement

Unsigned n-bit numbers represent values from 0 to $2^n - 1$. Signed binary systems use the most significant bit (MSB) to indicate the sign, but arithmetic is easiest with complement representations.

3.1 1's Complement

To form the 1's complement of a binary number, invert every bit: 0 becomes 1 and 1 becomes 0.

$$\text{1's complement of } N = \text{bitwise NOT of } N$$

Solved Example 3.1**Question: Find the 1's complement of 00101101.**

Original: 00101101

Invert bits: 11010010

Therefore the 1's complement is 11010010.

3.2 2's Complement

To form the 2's complement, take the 1's complement and add 1 to the least significant bit.

$$\text{2's complement of } N = (\text{1's complement of } N) + 1$$

Solved Example 3.2**Question: Find the 8-bit 2's complement representation of -45.**

+45 = 00101101.

1's complement = 11010010.

Add 1 => 11010011.

Hence -45 in 8-bit 2's complement is 11010011.

3.3 Range of n-bit 2's Complement Numbers

$$-2^{(n-1)} \text{ to } +(2^{(n-1)} - 1)$$

For 8 bits, the range is -128 to +127. The asymmetry exists because zero has only one representation in 2's complement.

3.4 Reading a Negative 2's Complement Number

If the MSB is 0, read the number normally. If the MSB is 1, the number is negative: take its 2's complement to find the magnitude, then attach a minus sign.

Solved Example 3.3**Question: Interpret 11100110 as an 8-bit 2's complement number.**

MSB=1, so it is negative.

Invert: 00011001; add 1: 00011010 = 26.

Therefore, 11100110 represents -26.

3.5 Subtraction Using 2's Complement

To calculate $A - B$ using fixed n-bit arithmetic: represent A and B using n bits, take the 2's complement of B, add it to A, and discard any carry beyond n bits. If the n-bit result has MSB 1, interpret it as a negative 2's complement number.

Solved Example 3.4**Question: Compute 27 - 19 using 8-bit 2's complement.**

27 = 00011011; 19 = 00010011.

2's complement of 19: 11101101.

00011011 + 11101101 = 1 00001000.

Discard the carry out. Result = 00001000 = 8.

Solved Example 3.5**Question: Compute 19 - 27 using 8-bit 2's complement.**

19 = 00010011; 27 = 00011011.

2's complement of 27: 11100101.

00010011 + 11100101 = 11111000.

MSB is 1. Complement 11111000: invert 00000111, add 1 => 00001000 = 8.

Therefore the result is -8.

3.6 Overflow in Signed Addition

Overflow occurs when the correct mathematical result cannot fit in the available number of bits. In 2's complement addition, overflow can occur when two numbers of the same sign are added and the result has the opposite sign.

Fast overflow rule

Positive + positive giving MSB 1 => overflow. Negative + negative giving MSB 0 => overflow. Adding numbers with different signs cannot cause signed overflow.

3.7 Why 2's Complement Is Preferred

Both 1's complement and 2's complement use bit inversion to form negative values, but 2's complement has major hardware advantages: it has only one zero, addition uses the same binary adder for positive and negative operands, and subtraction can be performed by addition of the 2's complement.

Feature	1's complement	2's complement
Negative of a number	Invert all bits	Invert all bits and add 1
Zero representations	Two: +0 and -0	One
n-bit range	$-(2^{(n-1)}-1)$ to $+(2^{(n-1)}-1)$	$-2^{(n-1)}$ to $+(2^{(n-1)}-1)$
Addition detail	End-around carry may be needed	Discard carry beyond n bits
Common computer arithmetic	Less common	Standard signed integer representation

3.8 Addition in 1's Complement

When adding 1's-complement signed numbers, any carry out of the MSB is added back to the LSB. This is called end-around carry. This rule is specific to 1's-complement arithmetic and should not be mixed with 2's-complement arithmetic.

Solved Example 3.6

Question: Using 4-bit 1's complement, add +5 and -3.

+5 = 0101. +3 = 0011, so -3 in 1's complement = 1100.

0101 + 1100 = 1 0001. There is a carry out of the MSB.

Add the end-around carry: 0001 + 1 = 0010 = +2.

Hence +5 + (-3) = +2.

3.9 Sign Extension and Fixed-Width Thinking

Binary signed arithmetic is always interpreted at a fixed width. When a 2's-complement number is widened, copy the sign bit into the new leading positions. This is sign extension. For example, 8-bit -5 is 11111011; as a 12-bit value it becomes 1111 11111011. Simply adding leading zeros would change the value and sign.

Solved Example 3.7

Question: Check overflow for 8-bit 2's-complement addition 100 + 50.

100 = 01100100, 50 = 00110010.

Sum = 10010110. The operands are both positive but the 8-bit result has sign bit 1.

The mathematical sum 150 is outside the 8-bit signed range -128 to +127, so overflow has occurred.

Important: the carry out and signed overflow are different concepts.

Edge case

In n-bit 2's complement, the most negative number $-2^{(n-1)}$ has no positive counterpart at the same width. Example: -128 exists in 8 bits, but +128 does not. Taking the 2's complement of 10000000 therefore produces 10000000 again.

4. Digital Codes: BCD, ASCII and Gray Code

4.1 BCD (8421 Binary Coded Decimal)

In 8421 BCD, each decimal digit is encoded separately using four bits. Only 0000 through 1001 are valid BCD digit codes; 1010 through 1111 are invalid for a single decimal digit.

Decimal digit	BCD	Decimal digit	BCD
0	0000	5	0101

Decimal digit	BCD	Decimal digit	BCD
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Solved Example 4.1**Question: Write decimal 59 in BCD.**

Encode each digit separately: 5 -> 0101, 9 -> 1001.

Therefore 59₁₀ = 0101 1001 (BCD).Do not convert 59 directly to binary; 59₁₀ = 111011₂, which is not the same representation as BCD.**Common mistake**

BCD represents decimal digits one by one. Pure binary represents the value as one binary number. They are generally different bit patterns.

4.2 ASCII

ASCII (American Standard Code for Information Interchange) is a character code used to represent letters, digits, punctuation and control characters. Standard ASCII uses 7 bits and can represent 128 codes (0-127).

Character	Decimal ASCII	Hex ASCII	7-bit binary
0	48	30	0110000
9	57	39	0111001
A	65	41	1000001
Z	90	5A	1011010
a	97	61	1100001
z	122	7A	1111010
Space	32	20	0100000

Useful pattern

ASCII digits 0-9 occupy consecutive codes 48-57; uppercase A-Z occupy 65-90; lowercase a-z occupy 97-122.

4.3 Gray Code

Gray code is a unit-distance code: consecutive code words differ in only one bit. This reduces ambiguity in applications where bits may not change at exactly the same instant, such as position encoders.

Binary to Gray Conversion

- The most significant Gray bit is the same as the most significant binary bit.
- Each next Gray bit is the XOR of two adjacent binary bits: $G_i = B_{(i+1)} \text{ XOR } B_i$.

Solved Example 4.2**Question: Convert binary 1011 to Gray code.** $B = 1\ 0\ 1\ 1.$ $G_3 = B_3 = 1.$ $G_2 = B_3 \text{ XOR } B_2 = 1 \text{ XOR } 0 = 1.$ $G_1 = B_2 \text{ XOR } B_1 = 0 \text{ XOR } 1 = 1.$ $G_0 = B_1 \text{ XOR } B_0 = 1 \text{ XOR } 1 = 0.$

Gray code = 1110.

Gray to Binary Conversion

- The most significant binary bit equals the most significant Gray bit.
- Each next binary bit equals the previous binary bit XOR the current Gray bit.

Solved Example 4.3**Question: Convert Gray code 1110 to binary.** $B_3 = G_3 = 1.$ $B_2 = B_3 \text{ XOR } G_2 = 1 \text{ XOR } 1 = 0.$ $B_1 = B_2 \text{ XOR } G_1 = 0 \text{ XOR } 1 = 1.$ $B_0 = B_1 \text{ XOR } G_0 = 1 \text{ XOR } 0 = 1.$

Binary = 1011.

4.4 BCD in More Detail

8421 BCD is a weighted code because the four positions have weights 8, 4, 2 and 1. The six 4-bit combinations 1010 through 1111 are unused for a single decimal digit. For a multi-digit number, keep a visible space between each 4-bit group so that digit boundaries are clear.

Decimal	Pure binary	8421 BCD
9	1001	1001
10	1010	0001 0000
25	11001	0010 0101
59	111011	0101 1001
407	110010111	0100 0000 0111

Observation

BCD and pure binary are identical only for single decimal digits 0-9. From decimal 10 onward, the encodings generally differ because BCD preserves decimal digit boundaries.

4.5 ASCII as a Character Representation

ASCII assigns a numeric code to a character; the bit pattern represents the code, not the visual shape of the character. Therefore character digit '5' and numeric value 5 are different concepts: ASCII '5' is decimal code 53 (0110101 in 7-bit ASCII), whereas binary integer 5 is 101.

Solved Example 4.4**Question: Find the 7-bit ASCII binary codes for the text "CAT".**

C is decimal 67 -> 1000011.

A is decimal 65 -> 1000001.

T is decimal 84 -> 1010100.

So the character sequence is represented by three separate ASCII code words: 1000011 1000001 1010100.

4.6 Gray Code Sequence and the Unit-Distance Property

Decimal	4-bit binary	4-bit Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Look down the Gray-code column: between every two consecutive rows exactly one bit changes. This is why Gray coding is useful in mechanical and optical position encoders, where simultaneous changes in several binary bits could momentarily produce an incorrect intermediate reading.

Exam distinction

BCD is a decimal-digit code; ASCII is a character code; Gray is a unit-distance code. They solve different representation problems and should not be treated as alternative forms of ordinary binary arithmetic.

5. Boolean Algebra Fundamentals

Boolean algebra deals with variables that take only two values: 0 and 1. In digital logic, 0 and 1 usually represent two voltage ranges. The principal operations are OR (+), AND (.) and NOT (overbar or prime).

Operation	Notation	Result rule
OR	$A + B$	1 if at least one input is 1
AND	$A.B$ or AB	1 only if all inputs are 1
NOT	A'	complements/inverts the input

5.1 Fundamental Postulates and Theorems

Law	OR form	AND form
Identity	$A + 0 = A$	$A.1 = A$
Null/Dominance	$A + 1 = 1$	$A.0 = 0$
Idempotent	$A + A = A$	$A.A = A$
Complement	$A + A' = 1$	$A.A' = 0$

Law	OR form	AND form
Involution	$(A')' = A$	$(A')' = A$
Commutative	$A+B = B+A$	$AB = BA$
Associative	$A+(B+C)=(A+B)+C$	$A(BC)=(AB)C$
Distributive	$A+BC=(A+B)(A+C)$	$A(B+C)=AB+AC$
Absorption	$A+AB=A$	$A(A+B)=A$
De Morgan	$(A+B)' = A'B'$	$(AB)' = A'+B'$

Principle of duality

A valid Boolean identity remains valid if OR and AND are interchanged and 0 and 1 are interchanged. This is why many laws appear in pairs.

5.2 Algebraic Simplification Examples

Solved Example 5.1

Question: Simplify $F = A + A'B$.

Use $A + A'B = (A + A')(A + B)$ by distributive law.

$= (1)(A+B) = A+B$.

Therefore $F = A + B$.

Solved Example 5.2

Question: Simplify $F = AB + AB' + A'B$.

$AB + AB' = A(B+B') = A$.

So $F = A + A'B = A + B$.

Therefore the simplified expression is $A+B$.

Solved Example 5.3

Question: Apply De Morgan's theorem to $(A + BC)'$.

$(A+BC)' = A' \cdot (BC)'$

$(BC)' = B' + C'$

Therefore $(A+BC)' = A'(B'+C')$.

5.3 Understanding Boolean Laws Instead of Memorising Them

Boolean algebra uses two-valued logic, so its addition and multiplication follow OR and AND rules rather than ordinary arithmetic. For example, $A+A=A$ and $A.A=A$. A useful test for any identity is to compare the conditions under which both sides become 1, or to verify the identity by a truth table.

Solved Example 5.4

Question: Verify the absorption law $A + AB = A$ by reasoning.

If $A=0$, then $A+AB=0+0=0$, which equals A .

If $A=1$, then $A+AB=1+B=1$, which again equals A .

Therefore the product AB is redundant whenever A is already present as an OR term.

5.4 A Systematic Algebraic Simplification Method

1. Remove constants using identity and dominance laws.
2. Look for complements such as $X+X'=1$ or $XX'=0$.
3. Factor common literals: $AB+AC=A(B+C)$.
4. Use absorption to remove redundant terms.

5. Apply De Morgan to complemented brackets one level at a time.
6. Stop when the expression has fewer terms/literals and no obvious law reduces it further.

Solved Example 5.5

Question: Simplify $F = A'B + AB + AB'$.

$$AB + AB' = A(B+B') = A.$$

$$\text{So } F = A'B + A.$$

$$\text{Using } A + A'B = A + B, \text{ the result is } F = A + B.$$

Solved Example 5.6

Question: Simplify $F = (A+B)(A+B')$.

$$\text{Use } (X+Y)(X+Z) = X + YZ. \text{ Therefore } F = A + BB'.$$

$$\text{Since } BB' = 0, F = A.$$

This result can also be obtained by expansion followed by absorption.

5.5 De Morgan on Multi-Level Expressions

When a complement covers a bracket, complement every literal and interchange AND with OR. Work from the outside inward. Brackets must be respected; complementing a product inside a larger sum requires a second De Morgan step.

Solved Example 5.7

Question: Simplify $Y = [(A+B')C]'$.

$$\text{Outer De Morgan: } Y = (A+B')' + C'.$$

$$\text{Then } (A+B')' = A'B.$$

$$\text{Therefore } Y = A'B + C'.$$

Exam technique

In a Boolean simplification question, write the law beside the important transformation steps. This makes your method visible and helps avoid accidental use of ordinary algebra rules.

6. Logic Gates and Truth Tables

A logic gate is a physical digital circuit that implements a Boolean operation. A truth table lists the output for every possible input combination.

6.1 Basic and Derived Gates

Gate	Boolean expression	Output is 1 when...
AND	$Y = AB$	both inputs are 1
OR	$Y = A + B$	at least one input is 1
NOT	$Y = A'$	input is 0
NAND	$Y = (AB)'$	not both inputs are 1
NOR	$Y = (A+B)'$	both inputs are 0
XOR	$Y = A'B + AB'$	inputs are different
XNOR	$Y = AB + A'B'$	inputs are the same

Two-Input Gate Truth Table

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NOT Gate Truth Table

A	A'
0	1
1	0

6.2 NAND and NOR as Universal Gates

A universal gate can be used alone to implement NOT, AND and OR, and therefore any Boolean function. Both NAND and NOR are universal.

Required function	Using NAND only	Using NOR only
NOT A	A NAND A = A'	A NOR A = A'
A AND B	First X=A NAND B, then Y=X NAND X	Use De Morgan construction from NOR gates
A OR B	Invert A and B with NAND; NAND the inverted inputs	First X=A NOR B, then Y=X NOR X

Solved Example 6.1

Question: Implement OR using NAND gates only.

By De Morgan, $A+B = (A'B')'$.

Generate A' using A NAND A and B' using B NAND B.

NAND these results: $Y = (A'B')' = A+B$.

Total NAND gates required = 3.

Exam writing tip

For a gate question, write the Boolean expression and truth table even if a circuit sketch is also required. These two items make the operation unambiguous.

6.3 Gate Interpretation and Useful Patterns

A truth table completely defines a combinational gate. AND behaves like an all-conditions detector, OR behaves like an any-condition detector, and NOT reverses a logic level. NAND and NOR are the complemented forms of AND and OR. XOR is especially important because it detects inequality for two bits; XNOR detects equality.

Gate	Useful verbal interpretation	Typical role
AND	All required inputs are 1	Enable/permission condition
OR	At least one input is 1	Combine alternative conditions
NOT	Reverse the logic value	Generate complement
XOR	Inputs differ	Sum/difference bit, parity/inequality
XNOR	Inputs are equal	Comparator equality stage

Gate	Useful verbal interpretation	Typical role
NAND/NOR	Complemented AND/OR	Universal implementation

6.4 Building Basic Gates Using NAND Only

$$\text{NOT } A = A \text{ NAND } A$$

$$A \text{ AND } B = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$$

$$A \text{ OR } B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$$

These identities prove universality: once NOT, AND and OR can be built, any Boolean expression can be implemented. The corresponding NOR-only constructions follow from the dual form of De Morgan's theorem.

6.5 Building Basic Gates Using NOR Only

$$\text{NOT } A = A \text{ NOR } A$$

$$A \text{ OR } B = (A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$$

$$A \text{ AND } B = (A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$$

Solved Example 6.2

Question: Show that XNOR is the equality detector for two inputs.

XNOR expression is $AB + A'B'$.

For 00, the second term is 1; for 11, the first term is 1.

For 01 and 10, neither product term is 1.

Hence $\text{XNOR}=1$ exactly when A and B are equal.

Common conceptual error

XOR and OR are not the same. They differ for input 11: $\text{OR}=1$ but $\text{XOR}=0$. XOR means "one or the other, but not both" for two inputs.

7. SOP, POS, Minterms and Maxterms

7.1 Product Term and Sum Term

- A product term is an AND of literals, e.g., $A'BC$.
- A sum term is an OR of literals, e.g., $A + B' + C$.
- A literal is a variable or its complement, e.g., A or A' .

7.2 Sum of Products (SOP)

In SOP form, multiple product terms are ORed together. Canonical SOP is a sum of minterms. A minterm contains every variable exactly once and has value 1 for exactly one truth-table row.

Minterm rule

For a truth-table row, write a variable uncomplemented when its value is 1 and complemented when its value is 0.

7.3 Product of Sums (POS)

In POS form, multiple sum terms are ANDed together. Canonical POS is a product of maxterms. A maxterm contains every variable exactly once and has value 0 for exactly one truth-table row.

Maxterm rule

For a truth-table row, write a variable uncomplemented when its value is 0 and complemented when its value is 1.

7.4 Standard Notation

$$F(A, B, C) = \Sigma m(1, 2, 5, 7) \quad \text{or} \quad F(A, B, C) = \Pi M(0, 3, 4, 6)$$

The minterm indices list the rows where $F=1$. The maxterm indices list the rows where $F=0$. For a complete truth table, the two sets are complementary.

Solved Example 7.1

Question: Write the canonical SOP for $F(A, B, C) = \Sigma m(1, 2, 5, 7)$.

$m_1 = 001 \Rightarrow A'B'C$

$m_2 = 010 \Rightarrow A'BC'$

$m_5 = 101 \Rightarrow AB'C$

$m_7 = 111 \Rightarrow ABC$

Therefore $F = A'B'C + A'BC' + AB'C + ABC$.

Solved Example 7.2

Question: For the same function, write canonical POS.

Rows not in the minterm list are 0, 3, 4, 6.

$M_0 = 000 \Rightarrow (A+B+C)$

$M_3 = 011 \Rightarrow (A+B'+C')$

$M_4 = 100 \Rightarrow (A'+B+C)$

$M_6 = 110 \Rightarrow (A'+B'+C)$

$F = \Pi M(0, 3, 4, 6) = (A+B+C)(A+B'+C')(A'+B+C)(A'+B'+C)$.

7.5 Canonical versus Standard Forms

An SOP expression is called canonical only when every product term is a minterm containing all variables. Likewise, a canonical POS contains only maxterms, each containing all variables. A simplified SOP such as $A+B'C$ is still SOP, but it is not canonical if the function has variables A, B and C because the term A does not contain B and C.

Form	Structure	Each term contains all variables?	Built from truth-table rows
Canonical SOP	OR of minterms	Yes	Rows where $F=1$
Simplified SOP	OR of product terms	Not necessarily	After simplification
Canonical POS	AND of maxterms	Yes	Rows where $F=0$
Simplified POS	AND of sum terms	Not necessarily	After simplification

7.6 From Truth Table to Sigma-m and Pi-M Notation

Number truth-table rows in ordinary binary order, with the leftmost variable treated as the most significant bit. Read the row number as a binary integer. Collect the row numbers for output 1 to obtain the minterm list; collect the output-0 rows to obtain the maxterm list.

Solved Example 7.3

Question: A 3-variable function is 1 for inputs 001, 010, 100 and 111. Write both canonical forms.

The 1-rows have decimal indices 1, 2, 4 and 7.

Therefore $F = \Sigma m(1, 2, 4, 7)$.

The remaining row indices are 0, 3, 5, 6, so $F = \Pi M(0, 3, 5, 6)$.

The two index sets are complementary because every truth-table row is either a 1-row or a 0-row.

7.7 Expanding a Non-Canonical Term into Minterms

If a product term is missing a variable, multiply it by $(X+X')=1$. Example for variables A, B, C: AB is independent of C, so $AB = AB(C+C') = ABC + ABC'$. This converts the term into two minterms without changing the function.

Solved Example 7.4

Question: Express $F=A+B'C$ as canonical SOP for variables A,B,C.

Expand $A = A(B+B')(C+C')$, producing minterms m_4, m_5, m_6, m_7 .

$B'C$ has A missing: $B'C(A+A')$ gives minterms m_1 and m_5 .

Combine duplicates: $F=\sum m(1,4,5,6,7)$.

Minterm/maxterm memory aid

For a minterm (a 1-row), bit 1 $\rightarrow$ uncomplemented variable and bit 0 $\rightarrow$ complemented variable. For a maxterm (a 0-row), the rule reverses: bit 0 $\rightarrow$ uncomplemented, bit 1 $\rightarrow$ complemented.

8. Karnaugh Maps (K-Maps): 2 to 4 Variables

A Karnaugh map is a graphical method for simplifying Boolean expressions. Adjacent cells differ in only one variable because row and column labels are arranged in Gray-code order.

8.1 Grouping Rules

- Place 1s in the cells corresponding to minterms for SOP simplification. For POS, place/use 0s corresponding to maxterms.
- Make rectangular groups containing 1, 2, 4, 8, ... cells only (powers of two).
- Make each group as large as possible. Larger groups eliminate more variables.
- Every required 1 must be covered at least once. Overlap is allowed when it helps make larger groups.
- Opposite edges are adjacent: left wraps to right, and top wraps to bottom.
- Diagonal cells are not adjacent.
- In an SOP group, keep only variables whose values stay constant across the group.

8.2 2-Variable K-Map Layout

A\B	0	1
0	m0	m1
1	m2	m3

8.3 3-Variable K-Map Layout

A\BC	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6

8.4 4-Variable K-Map Layout

AB\CD	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

Why Gray-code order?

The sequence 00, 01, 11, 10 changes only one bit at a time. That makes adjacent K-map cells differ in only one variable, which is the basis of simplification.

8.5 Solved K-Map Examples

Solved Example 8.1 - 2 variables

Question: Simplify $F(A,B)=\Sigma m(1,2,3)$.

Place 1s at m1, m2 and m3.

Group m2,m3: A is constant 1 $\Rightarrow$ term A.

Group m1,m3: B is constant 1 $\Rightarrow$ term B.

Therefore $F=A+B$.

Solved Example 8.2 - 3 variables

Question: Simplify $F(A,B,C)=\Sigma m(1,3,5,7)$.

The four minterms are exactly the cells where $C=1$.

They form one group of four.

A and B change within the group; C remains 1.

Therefore $F=C$.

Solved Example 8.3 - 3 variables

Question: Simplify $F(A,B,C)=\Sigma m(0,2,4,6,7)$.

Group m0,m2,m4,m6 as a four-cell wrap-around group: C remains 0 $\Rightarrow$ C' .

Group m6,m7 as a pair: A=1 and B=1 remain constant $\Rightarrow$ AB.

Therefore $F=C' + AB$.

Solved Example 8.4 - 4 variables

Question: Simplify $F(A,B,C,D)=\Sigma m(0,2,5,7,8,10,13,15)$.

Group m0,m2,m8,m10 as a four-cell group using edge adjacency: B=0 and D=0 remain constant $\Rightarrow$ $B'D'$.

Group m5,m7,m13,m15 as a four-cell group: B=1 and D=1 remain constant $\Rightarrow$ BD.

Therefore $F=B'D' + BD$ (XNOR of B and D).

8.6 POS Simplification with K-Map

For POS, group 0s instead of 1s. Within a 0-group, a variable that remains 0 appears uncomplemented in the resulting sum term, while a variable that remains 1 appears complemented.

Solved Example 8.5 - POS

Question: Simplify $F(A,B)=\Pi m(0,2)$.

Zeros are at m0(00) and m2(10). They form a vertical pair.

B remains 0 while A changes.

For a zero-group in POS, constant B=0 gives sum term (B).

Therefore $F=B$.

K-map self-check

After simplification, test the simplified expression for at least one original 1-row and one original 0-row. This catches many grouping or complement mistakes.

8.7 Why Grouping Simplifies an Expression

Two adjacent minterms differ in exactly one variable. When they are ORed, that changing variable disappears: $A'BC + ABC = BC(A'+A)=BC$. A group of 2 removes one variable, a group of 4 removes two variables, and a group of 8 removes three variables. This is the algebraic reason behind K-map grouping.

Group size	Variables eliminated	A term from a 4-variable map can have
1	0	4 literals
2	1	3 literals
4	2	2 literals
8	3	1 literal
16	4	constant 1

8.8 Essential K-Map Techniques

- Use the largest legal groups first; a larger group normally gives fewer literals.
- A cell may belong to more than one group. Overlap is legal and often necessary.
- Four corner cells of a 4-variable map are adjacent through both horizontal and vertical wrap-around.
- A group may wrap across the left/right edge or top/bottom edge, but it may not wrap diagonally.
- Every required 1 must be covered for SOP. It is not necessary to use every possible group.
- For POS repeat the same geometry using 0s, then apply the POS complement rule.

Solved Example 8.6 - 4-variable corner group

Question: Simplify $F(A,B,C,D)=\sum m(0,2,8,10)$.

The four 1s occupy the four corners when the K-map is drawn in Gray order.

All four corners form one wrap-around group of four.

Across these cells $B=0$ and $D=0$ remain constant; A and C change.

Therefore $F = B'D'$.

Solved Example 8.7 - overlap

Question: Simplify $F(A,B,C)=\sum m(1,3,5,6,7)$.

Group $m1, m3, m5, m7$ as a group of four: $C=1$ remains constant $\rightarrow C$.

The uncovered $m6$ can pair with $m7$: $A=1$ and $B=1$ remain constant $\rightarrow AB$.

Cell $m7$ is intentionally used in both groups; overlap is legal.

Therefore $F = C + AB$.

8.9 A Complete 4-Variable SOP Example

Solved Example 8.8

Question: Simplify $F(A,B,C,D)=\sum m(0,1,2,3,5,7,8,9,10,11)$.

Minterms $0,1,2,3,8,9,10,11$ form an 8-cell group covering rows $AB=00$ and $AB=10$. Here $B=0$ is constant $\rightarrow B'$.

Minterms $1,3,5,7$ form a 4-cell group. Here $A=0$ and $D=1$ remain constant $\rightarrow A'D$.

The first group already covers $m1, m3$, but overlap allows the second group to also cover them so that $m5, m7$ can be included in a group of four.

Hence $F = B' + A'D$.

8.10 POS K-Map Example with Four Variables

Solved Example 8.9

Question: Simplify $F(A,B,C,D)=\prod M(0,1,4,5,10,11,14,15)$ in POS form.

Group zeros $m0, m1, m4, m5$ as a 4-cell block: $A=0$ and $C=0$ remain constant.

For a zero-group, constant 0 appears uncomplemented, giving $(A+C)$.

Group zeros $m10, m11, m14, m15$: $A=1$ and $C=1$ remain constant.

For a zero-group, constant 1 appears complemented, giving $(A'+C')$.

Therefore $F=(A+C)(A'+C')$.

8.11 Common K-Map Failure Modes

Mistake	Why it is wrong	Correction
Using 00,01,10,11 order	Adjacent cells may differ in two bits	Use Gray order 00,01,11,10.
Group of 3 or 6	Does not eliminate variables cleanly	Only 1,2,4,8,... cells.
Diagonal grouping	Diagonal cells differ in more than one bit	Only horizontal/vertical adjacency.
Tiny groups first	Produces more literals than necessary	Prefer largest legal groups.
Wrong POS complement rule	Gives complement of required term	For 0-group: 0 -> uncomplemented, 1 -> complemented.

Exam workflow for a K-map

1) Write the minterm/maxterm list. 2) Draw Gray-labelled map. 3) Fill 1s or 0s. 4) Mark largest groups. 5) Write one simplified term per group. 6) Combine terms. 7) Verify one 1-row and one 0-row.

9. Combinational Logic Design

A combinational circuit has no memory: at any instant, its output depends only on the current input values. Examples in this unit include adders, subtractors, multiplexers, demultiplexers, encoders, decoders and comparators.

General Design Procedure

1. Understand the problem and identify input/output variables.
2. Construct the truth table.
3. Write Boolean expressions for each output.
4. Simplify the expressions using Boolean algebra or K-maps.
5. Implement using appropriate gates or standard building blocks.
6. Verify the circuit against the truth table.

Combinational vs sequential

Combinational: output depends only on present input. Sequential: output also depends on stored past state. Sequential circuits are covered in Unit II.

10. Adders

10.1 Half Adder

A half adder adds two one-bit inputs A and B. It produces Sum (S) and Carry (C). It does not accept a carry-in from a previous stage.

A	B	Sum S	Carry C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = A \oplus B$$

$$C = AB$$

10.2 Full Adder

A full adder adds A, B and an input carry C_{in} . It produces Sum S and output carry C_{out} .

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = A \text{ XOR } B \text{ XOR } C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in} = AB + C_{in}(A \text{ XOR } B)$$

10.3 Full Adder Using Two Half Adders

- Half Adder 1 adds A and B: $S_1 = A \text{ XOR } B$, $C_1 = AB$.
- Half Adder 2 adds S_1 and C_{in} : $S = S_1 \text{ XOR } C_{in}$, $C_2 = S_1 \cdot C_{in}$.
- Final carry: $C_{out} = C_1 + C_2$.

Design interpretation

A full adder can be constructed using two half adders and one OR gate.

Solved Example 10.1

Question: For $A=1$, $B=1$, $C_{in}=1$, find full-adder outputs.

$$S = 1 \text{ XOR } 1 \text{ XOR } 1 = 1.$$

$$C_{out} = AB + AC_{in} + BC_{in} = 1+1+1 = 1 \text{ (Boolean OR).}$$

The binary addition is $1+1+1 = 11_2$, so $S=1$ and $C_{out}=1$.

10.4 Deriving the Full-Adder Equations

From the truth table, Sum is 1 when an odd number of the three inputs is 1. That is exactly the parity behaviour of XOR, so $S = A \text{ XOR } B \text{ XOR } C_{in}$. Carry-out is 1 whenever at least two of the three input bits are 1, giving the majority expression $AB + AC_{in} + BC_{in}$.

Solved Example 10.2

Question: Derive C_{out} in the alternative form $AB + C_{in}(A \text{ XOR } B)$.

Start with $C_{out} = AB + AC_{in} + BC_{in}$.

Write $AC_{in} + BC_{in} = C_{in}(A+B)$.

The equivalent adder form $AB + C_{in}(A \text{ XOR } B)$ is obtained by separating the case $A=B=1$ into AB ; when A and B differ, $A \text{ XOR } B=1$ and C_{in} determines the carry.

This form directly matches the two-half-adder implementation: $C_1=AB$ and $C_2=C_{in}(A \text{ XOR } B)$.

10.5 Cascading Full Adders for Multi-Bit Addition

To add two multi-bit binary numbers, use one full-adder stage per bit position. The carry-out from a lower position becomes the carry-in of the next higher position. This arrangement is called a ripple-carry adder because the carry can propagate through successive stages.

Solved Example 10.3

Question: Add 1011 and 0110 using bit-by-bit carry propagation.

LSB: $1+0 \rightarrow$ sum 1, carry 0.

Next: $1+1+0 \rightarrow$ sum 0, carry 1.

Next: $0+1+1 \rightarrow$ sum 0, carry 1.

MSB: $1+0+1 \rightarrow$ sum 0, carry 1.

Final result = 1 0001 = 17₁₀, matching 11+6.

Minor-exam derivation tip

For a full-adder design question, a strong answer contains: truth table, canonical expression or K-map reasoning, simplified S and Cout expressions, and the two-half-adder realization.

11. Subtractors

11.1 Half Subtractor

A half subtractor subtracts B from A and produces Difference (D) and Borrow (Bout).

A	B	Difference D	Borrow Bout
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$D = A \text{ XOR } B$$

$$\text{Bout} = A'B$$

11.2 Full Subtractor

A full subtractor subtracts B and borrow-in Bin from A. It produces Difference D and borrow-out Bout.

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = A \text{ XOR } B \text{ XOR } \text{Bin}$$

$$\text{Bout} = A'B + A'\text{Bin} + B.\text{Bin}$$

Solved Example 11.1

Question: For A=0, B=1, Bin=1, find the full-subtractor outputs.

We need $0 - 1 - 1$. A borrow is required.

From the truth table, D=0 and Bout=1.

Expression check: $D=0 \text{ XOR } 1 \text{ XOR } 1=0$; Bout=1.

Adder vs subtractor memory aid

Half adder carry = AB . Half subtractor borrow = $A'B$. Sum and difference are both XOR for two inputs.

11.3 Understanding the Borrow Condition

A borrow is needed when the amount being subtracted from a bit position is larger than the available minuend bit. In a half subtractor, this happens only for $A=0, B=1$, hence Borrow= $A'B$. In a full subtractor the incoming borrow must also be considered, which creates three conditions that can require a borrow-out.

11.4 Full Subtractor Using Two Half Subtractors

- Half Subtractor 1 subtracts B from A: $D1=A \text{ XOR } B$, $B1=A'B$.
- Half Subtractor 2 subtracts Bin from D1: $D=D1 \text{ XOR } \text{Bin}$, $B2=D1' \cdot \text{Bin}$.
- Final borrow is $\text{Bout}=B1+B2$.
- After simplification this is equivalent to $\text{Bout}=A'B + A'\text{Bin} + B \text{ Bin}$.

Solved Example 11.2

Question: Subtract 0101 from 1001 using binary borrowing.

1001₂ is 9 and 0101₂ is 5, so the expected result is 4.

Working bit by bit from the LSB gives $1001 - 0101 = 0100$.

The result 0100₂ = 4 confirms the subtraction.

For computer hardware, the same subtraction can often be performed by adding the 2's complement of the subtrahend rather than using a separate multi-bit subtractor path.

Connection to signed arithmetic

The full subtractor is a direct combinational building block for bit-by-bit subtraction. A CPU may instead reuse an adder by complementing B and setting an initial carry-in of 1, implementing $A + (2\text{'s complement of } B)$. Both viewpoints are useful.

12. Multiplexers (MUX)

A multiplexer is a many-to-one data selector. It selects one of several input lines and connects it to a single output according to select inputs.

12.1 2-to-1 Multiplexer

Select S	Output Y
0	I0
1	I1

$$Y = S'I_0 + SI_1$$

12.2 4-to-1 Multiplexer

S1	S0	Selected input
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$$Y = S1'S0'I_0 + S1'S0 I_1 + S1 S0'I_2 + S1 S0 I_3$$

12.3 Implementing a Boolean Function Using a MUX

Choose some variables as select inputs. For each select combination, express the required output in terms of the remaining variable(s) and connect the corresponding MUX data input.

Solved Example 12.1

Question: Implement $F(A,B,C)=\sum m(1,2,6,7)$ using a 4:1 MUX with A,B as select inputs.

Use $S_1=A$, $S_0=B$. Check F for each AB combination as C varies.

AB=00: rows m_0, m_1 -> outputs 0,1 => $I_0=C$.

AB=01: rows m_2, m_3 -> outputs 1,0 => $I_1=C'$.

AB=10: rows m_4, m_5 -> outputs 0,0 => $I_2=0$.

AB=11: rows m_6, m_7 -> outputs 1,1 => $I_3=1$.

Connect $I_0=C$, $I_1=C'$, $I_2=0$, $I_3=1$.

13. Demultiplexers (DEMUX)

A demultiplexer is a one-to-many data distributor. One data input is routed to one of several outputs according to select inputs.

13.1 1-to-4 Demultiplexer

S1	S0	Active output (receives D)
0	0	Y0
0	1	Y1
1	0	Y2
1	1	Y3

$$Y_0 = D \cdot S_1' \cdot S_0' \quad Y_1 = D \cdot S_1' \cdot S_0 \quad Y_2 = D \cdot S_1 \cdot S_0' \quad Y_3 = D \cdot S_1 \cdot S_0$$

MUX vs DEMUX

MUX selects one of many inputs to one output. DEMUX routes one input to one of many outputs. Select lines determine the chosen path in both.

12.4 General Multiplexer Relationships

A multiplexer with n select lines can select among 2^n data inputs. Thus a 2:1 MUX needs 1 select line, a 4:1 needs 2, an 8:1 needs 3, and a 16:1 needs 4. Each data input is ANDed with the select-line minterm that chooses it, and all such products are ORed together.

MUX	Data inputs	Select lines	Output lines
2:1	2	1	1
4:1	4	2	1
8:1	8	3	1
16:1	16	4	1

12.5 MUX as a Boolean Function Generator

A MUX can implement a truth table because its select inputs choose one data entry for each select combination. If all variables are used as select inputs, connect each data input directly to 0 or 1 according to the function value. If one variable is left as a data variable, the required data inputs may become 0, 1, X or X'.

Solved Example 12.2

Question: Implement $F(A,B,C)=\Sigma m(0,1,6,7)$ using a 4:1 MUX with A,B as select lines.

AB=00 covers m_0, m_1 ; both are 1 $\rightarrow I_0=1$.

AB=01 covers m_2, m_3 ; both are 0 $\rightarrow I_1=0$.

AB=10 covers m_4, m_5 ; both are 0 $\rightarrow I_2=0$.

AB=11 covers m_6, m_7 ; both are 1 $\rightarrow I_3=1$.

So connect $I_0=1, I_1=0, I_2=0, I_3=1$. The output is independent of C.

13.2 DEMUX Operation in More Detail

A DEMUX contains one data input and several output paths. For any select code, only the selected path receives the data value D; unselected outputs remain 0. If D is fixed at 1, a DEMUX behaves like a decoder because the select inputs activate exactly one output line.

Solved Example 13.1

Question: For a 1:4 DEMUX, $D=1$ and $S_1S_0=10$. Find Y_0 - Y_3 .

Select code 10 chooses output Y_2 .

$Y_2=D=1$. All other outputs are forced to 0.

Thus $Y_0Y_1Y_2Y_3 = 0010$ when listed in order Y_0, Y_1, Y_2, Y_3 .

Where these blocks are used

MUX: choose one data source for a shared line. DEMUX: route one source to one destination. Decoder: activate one line from a binary address. Their equations look related because all are built from select-variable minterms.

14. Encoders and Decoders**14.1 Encoder**

An encoder converts one active input line into a binary code. A basic 4-to-2 encoder assumes that exactly one input is active at a time.

Active input	Y_1	Y_0
D0	0	0
D1	0	1
D2	1	0
D3	1	1

$$Y_1 = D_2 + D_3$$

$$Y_0 = D_1 + D_3$$

Limitation of a basic encoder

If more than one input is 1 simultaneously, the output may be ambiguous. Priority encoders solve this by assigning precedence, but the basic encoder is the core syllabus concept.

14.2 Decoder

A decoder performs the reverse type of operation: it converts an n-bit binary input into one of up to 2^n active output lines. A 2-to-4 decoder has inputs A,B and outputs Y_0 - Y_3 .

A	B	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0

A	B	Y0	Y1	Y2	Y3
1	0	0	0	1	0
1	1	0	0	0	1

$$Y0 = A'B' \quad Y1 = A'B \quad Y2 = AB' \quad Y3 = AB$$

14.3 Function Realization Using a Decoder

Each decoder output corresponds to a minterm. Therefore a Boolean function in sum-of-minterms form can be realized by ORing the required decoder outputs.

Solved Example 14.1

Question: Implement $F(A,B) = \sum m(1,2,3)$ using a 2-to-4 decoder.

Decoder outputs correspond to m_0, m_1, m_2 and m_3 .

F is 1 for m_1, m_2, m_3 .

Therefore connect Y_1, Y_2, Y_3 to an OR gate: $F = Y_1 + Y_2 + Y_3$.

14.4 Encoder Equations and Ambiguity

For the 4-to-2 encoder, Y_1 must be 1 when D_2 or D_3 is active, while Y_0 must be 1 when D_1 or D_3 is active. These observations directly produce $Y_1 = D_2 + D_3$ and $Y_0 = D_1 + D_3$. The equations assume one-hot input: exactly one data input is 1.

Solved Example 14.2

Question: Why is a basic 4-to-2 encoder ambiguous if D_1 and D_2 are both 1?

$Y_1 = D_2 + D_3$ becomes 1.

$Y_0 = D_1 + D_3$ also becomes 1.

The output 11 normally indicates D_3 , even though D_3 is actually 0.

Therefore the basic encoder cannot uniquely represent multiple active inputs; a priority rule is needed in that situation.

14.5 3-to-8 Decoder

A 3-to-8 decoder has three input variables and eight outputs. Each output represents one 3-variable minterm. This makes it a direct hardware realization of the complete minterm set for three variables.

ABC	Active output	Minterm
000	Y0	$A'B'C'$
001	Y1	$A'B'C$
010	Y2	$A'BC'$
011	Y3	$A'BC$
100	Y4	$AB'C'$
101	Y5	$AB'C$
110	Y6	ABC'
111	Y7	ABC

Solved Example 14.3

Question: Using a 3-to-8 decoder, realize $F(A,B,C) = \sum m(1,3,5,6)$.

Feed A, B, C to the decoder inputs.

The required minterms are available directly as Y_1, Y_3, Y_5, Y_6 .

OR these outputs: $F = Y_1 + Y_3 + Y_5 + Y_6$.

14.6 Enable Input - Practical Decoder Idea

Many practical decoders include an enable input. When the decoder is disabled, none of the outputs is selected; when enabled, normal decoding occurs. Enable inputs are useful when several decoder blocks are combined or when address decoding must be active only under a specific condition. The exact active-high or active-low convention depends on the device.

Encoder versus decoder

Encoder compresses one active line into a shorter binary code. Decoder expands a binary code into one active line. They are conceptually opposite operations, although real devices may add enable or priority features.

15. Magnitude Comparators

A magnitude comparator compares two binary numbers and indicates whether $A > B$, $A = B$ or $A < B$. The three outputs are mutually exclusive for valid inputs.

15.1 One-Bit Comparator

A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

$$A > B = AB' \quad A = B = A'B' + AB \quad A < B = A'B$$

15.2 Multi-Bit Comparison Idea

For multi-bit numbers, compare the most significant bits first. If they differ, they decide the result. Only when the MSBs are equal do we compare the next lower bit, and so on.

Solved Example 15.1

Question: Compare $A=1011$ and $B=1001$.

Compare from MSB: first bit $1=1$; second bit $0=0$; third bit $1>0$.

The remaining lower bit cannot change the decision.

Therefore $A > B$ ($11_{10} > 9_{10}$).

Hardware intuition

A comparator is essentially a cascade of equality checks and greater/less decisions beginning at the most significant bit.

15.3 Deriving a Two-Bit Comparator

Let $A=A_1A_0$ and $B=B_1B_0$. The MSBs A_1 and B_1 have priority. A is greater immediately if $A_1=1$ and $B_1=0$. If the MSBs are equal, the LSBs decide. Define $E_1=A_1 \text{ XNOR } B_1$ and $E_0=A_0 \text{ XNOR } B_0$.

$$\begin{aligned} A = B &: E_1 E_0 \\ A > B &: A_1 B_1' + E_1 A_0 B_0' \\ A < B &: A_1' B_1 + E_1 A_0' B_0 \end{aligned}$$

These expressions illustrate the general comparison principle: a lower bit matters only when every more significant bit is equal. Larger comparators repeat the same cascade idea.

Solved Example 15.2**Question: Compare A=10 and B=01 using the two-bit logic.**

At the MSB, $A_1=1$ and $B_1=0$, so the term A_1B_1' is already 1.

Therefore $A>B$ regardless of the lower bits.

Numerically, $10_2=2$ and $01_2=1$, confirming the comparator output.

15.4 Comparator Output Properties

- For valid binary inputs exactly one of $A>B$, $A=B$ and $A<B$ is 1.
- Equality can be built from XNOR at each bit followed by AND of all equality results.
- Greater-than and less-than decisions proceed from MSB toward LSB.
- Comparator circuits are used in sorting/control decisions, address checks and arithmetic status logic.

Exam-ready answer structure

For a comparator question: define the three outputs, write the 1-bit truth table and expressions, then explain MSB-first comparison for multi-bit numbers. If asked to derive 2-bit logic, explicitly gate the LSB decision with MSB equality.

16. Integrated Solved Problems**Problem 16.1****Question: Convert (725)₈ to binary and hexadecimal.**

Octal to binary: $7=111$, $2=010$, $5=101 \Rightarrow 111010101_2$.

Group into 4 bits for hex: $0001\ 1101\ 0101 \Rightarrow 1D5_{16}$.

Answer: $(725)_8 = (111010101)_2 = (1D5)_{16}$.

Problem 16.2**Question: Find the 8-bit 2's complement representation of -73.**

$73_{10} = 01001001_2$.

Invert $\Rightarrow 10110110$; add 1 $\Rightarrow 10110111$.

Answer: $-73 = 10110111$ in 8-bit 2's complement.

Problem 16.3**Question: For $F(A,B,C)=\Sigma m(0,2,3,6,7)$, obtain a simplified SOP using K-map.**

Group m_2, m_3, m_6, m_7 as a group of four: B remains 1 $\Rightarrow B$.

Group m_0, m_2 as a pair: $A=0$ and $C=0$ remain constant $\Rightarrow A'C'$.

Therefore $F = B + A'C'$.

Problem 16.4**Question: Design a circuit whose output is 1 when exactly one of A and B is 1.**

Truth table gives output 1 for 01 and 10.

Canonical SOP = $A'B + AB'$.

This is exactly XOR: $F=A \text{ XOR } B$.

Use one XOR gate.

Problem 16.5**Question: A 4:1 MUX has $I_0=0$, $I_1=1$, $I_2=1$, $I_3=0$. Find the output function in terms of S_1, S_0 .**

$Y=S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$.

Substitute data values: $Y=S_1'S_0 + S_1S_0'$.

Therefore $Y=S_1 \text{ XOR } S_0$.

Problem 16.6

Question: Using a 2-to-4 decoder, realize $F(A,B)=A+B$.

$A+B$ is 1 for input combinations 01,10,11 $\Rightarrow$ minterms 1,2,3.

So $F=\Sigma m(1,2,3)$.

OR decoder outputs Y_1, Y_2 and Y_3 .

Problem 16.7

Question: Design a 3-input majority function: output 1 when at least two of A,B,C are 1.

The output is 1 for 011,101,110,111 $\rightarrow F=\Sigma m(3,5,6,7)$.

K-map grouping gives terms AB, AC and BC .

Therefore $F=AB+AC+BC$.

This is the same form as the carry-out of a full adder, because carry occurs when at least two of A,B,C_{in} are 1.

Problem 16.8

Question: Implement $F(A,B,C)=A'C+BC$ using a 3-to-8 decoder.

Expand $A'C$: it is 1 for $A=0, C=1$ with either $B \rightarrow$ minterms m_1, m_3 .

BC is 1 for $B=1, C=1$ with either $A \rightarrow$ minterms m_3, m_7 .

Combine duplicates: $F=\Sigma m(1,3,7)$.

OR decoder outputs Y_1, Y_3, Y_7 .

17. One-Page Concept Revision

Topic	Must remember
Base-r value	Digit at position i contributes $\text{digit} * r^i$.
Binary-octal	Group 3 bits per octal digit.
Binary-hex	Group 4 bits per hex digit.
1's complement	Invert all bits.
2's complement	Invert all bits and add 1.
n-bit signed range	$-2^{(n-1)} \text{ to } +2^{(n-1)}-1$.
BCD	Encode each decimal digit separately using 4 bits.
Gray	Adjacent codes differ by one bit; binary $\rightarrow$ Gray uses adjacent XOR.
De Morgan	$(A+B)'=A'B'$ and $(AB)'=A'+B'$.
Canonical SOP	Sum of minterms for rows where $F=1$.
Canonical POS	Product of maxterms for rows where $F=0$.
K-map	Group powers of two; largest groups first; wrap-around allowed.
Half adder	$S=A \text{ XOR } B, C=AB$.
Full adder	$S=A \text{ XOR } B \text{ XOR } C_{in}; C_{out}=AB+AC_{in}+BC_{in}$.
Half subtractor	$D=A \text{ XOR } B, \text{Borrow}=A'B$.
MUX	Many inputs $\rightarrow$ one output.
DEMUX	One input $\rightarrow$ many outputs.
Encoder	One active input $\rightarrow$ binary code.
Decoder	Binary code $\rightarrow$ one active output.
Comparator	Produces $A>B, A=B, A<B$ outputs.

18. Common Mistakes to Avoid

- Reading repeated-division remainders from top to bottom instead of bottom to top.
- Forgetting that BCD encodes each decimal digit separately.
- Using ordinary binary range 0 to 2^n-1 for signed 2's complement questions.
- Confusing minterm and maxterm complement rules.
- Using binary order 00,01,10,11 on K-map columns instead of Gray order 00,01,11,10.
- Grouping diagonal K-map cells or making groups of 3, 6, etc. instead of powers of two.
- Forgetting wrap-around adjacency on K-map edges.
- Writing a full-adder truth table without C_{in} or a full-subtractor table without B_{in} .
- Interchanging MUX and DEMUX directions.
- Calling XOR a universal gate. NAND and NOR are the universal gates in this syllabus.

19. Minor-Exam Question Bank

Use these as self-test questions. Try them without notes first, then check the answer cues in the next section.

A. Very Short / Concept Questions

1. Define radix and positional number system.
2. Why is hexadecimal convenient for representing binary data?
3. Differentiate pure binary and BCD representation.
4. State the range of an n-bit 2's complement number.
5. Why is Gray code called a unit-distance code?
6. State both De Morgan theorems.
7. What is a minterm? What is a maxterm?
8. Why are NAND and NOR called universal gates?
9. What is wrap-around adjacency in a K-map?
10. Differentiate a MUX and DEMUX.
11. State one limitation of a basic encoder.
12. What three outputs does a magnitude comparator produce?

B. Numerical / Design Questions

1. Convert $(753.4)_{10}$ to binary and hexadecimal.
2. Convert $(9B.6)_{16}$ to binary and decimal.
3. Represent -91 in 8-bit 2's complement.
4. Perform 52-77 using 8-bit 2's complement arithmetic.
5. Encode decimal 407 in BCD.
6. Convert binary 110101 to Gray and convert the result back to binary.
7. Simplify $F = A + A'B + AB$ using Boolean algebra.
8. Write canonical SOP and POS for a 3-variable truth table with $F=1$ at rows 0,2,6,7.
9. Simplify $F(A,B,C) = \sum m(0,1,2,3,6,7)$ using K-map.
10. Simplify $F(A,B,C,D) = \sum m(0,1,2,3,8,9,10,11)$ using K-map.
11. Derive Sum and Carry expressions of a full adder.
12. Derive Difference and Borrow expressions of a half subtractor.
13. Implement $F(A,B,C) = \sum m(0,2,5,7)$ using a 4:1 MUX.
14. Use a 3-to-8 decoder concept to realize $F(A,B,C) = \sum m(1,3,5,6)$.
15. Write the truth table and expressions for a one-bit comparator.

20. Answer Cues for the Practice Bank

A. Concept Cues

Question	Cue
Radix	Number of distinct digit symbols; positional value depends on digit position and powers of the base.
Hex convenience	Each hex digit maps exactly to 4 binary bits, making long binary strings compact.
Binary vs BCD	Pure binary encodes the whole value; BCD separately encodes each decimal digit.
2's complement range	$-2^{(n-1)}$ to $+2^{(n-1)}-1$.
Gray code	Consecutive code words differ in exactly one bit.
De Morgan	$(A+B)'=A'B'$; $(AB)'=A'+B'$.
Minterm/maxterm	Minterm is 1 for one row; maxterm is 0 for one row.
Universal gate	NAND or NOR alone can create NOT, AND and OR.
Wrap-around	Opposite K-map edges are adjacent.
MUX/DEMUX	MUX many-to-one; DEMUX one-to-many.
Encoder limitation	Multiple simultaneous active inputs cause ambiguity in a basic encoder.
Comparator	$A>B$, $A=B$ and $A<B$.

B. Numerical / Design Final Answers

No.	Final answer / cue
1	$753.4_{10} = 111101011.100_2 = 1EB.8_{16}$.
2	$9B.6_{16} = 10011011.0110_2 = 155.375_{10}$.
3	$-91_{10} (8\text{-bit}) = 10100101$.
4	$52-77 = -25$; 8-bit result 11100111.
5	$407_{10} \text{ BCD} = 0100\ 0000\ 0111$.
6	$110101_2 \rightarrow \text{Gray } 101111$; converting back gives 110101.
7	$A + A'B + AB = A+B$.
8	$SOP = \sum m(0,2,6,7)$; $POS = \prod M(1,3,4,5)$.
9	$F = A' + B$.
10	$F = B'$.
11	$S = A \oplus B \oplus C_{in}$; $C_{out} = AB + AC_{in} + BC_{in}$.
12	$D = A \oplus B$; $Borrow = A'B$.
13	With A,B as selects: $I_0 = C'$, $I_1 = C'$, $I_2 = C$, $I_3 = C$.
14	OR decoder outputs Y_1, Y_3, Y_5, Y_6 .
15	$A>B = AB'$; $A=B = A'B' + AB$; $A<B = A'B$.

21. Last-Minute Revision Sheet

Conversions

Base $\rightarrow$ decimal: weighted sum. Decimal integer $\rightarrow$ base: repeated division, read remainders upward. Decimal fraction $\rightarrow$ base: repeated multiplication. Binary $\leftrightarrow$ octal: 3-bit groups. Binary $\leftrightarrow$ hex: 4-bit groups.

Complements

1's complement = invert. 2's complement = invert + 1. n-bit signed range = $-2^{(n-1)}$ to $+2^{(n-1)}-1$. For subtraction $A-B$, add A to the 2's complement of B.

Boolean + K-map

Remember identity, complement, absorption, distributive and De Morgan laws. SOP uses minterms/1s; POS uses maxterms/0s. K-map group sizes: 1,2,4,8,...; use Gray order 00,01,11,10; edges wrap.

Combinational circuits

Half adder: $S=A \oplus B$, $C=AB$. Full adder: $S=A \oplus B \oplus C_{in}$, $C_{out}=AB+AC_{in}+BC_{in}$. Half subtractor: $D=A \oplus B$, $Borrow=A'B$. MUX=many-to-one; DEMUX=one-to-many; encoder=input-to-code; decoder=code-to-output; comparator gives $>, =, <$.

Final 30-Minute Checklist

- Do one decimal/binary/octal/hex conversion in each direction.
- Do one 2's complement subtraction where the answer is negative.
- Recite De Morgan laws and universal-gate realizations.
- Solve one 3-variable and one 4-variable K-map.
- Write half/full adder and half/full subtractor truth tables from memory.
- Write 4:1 MUX and 2:4 decoder selection/truth tables.
- Check that you can explain encoder, decoder, comparator, MUX and DEMUX in one sentence each.

End of Unit I Notes

AR107 - Basics of Computer Organization | Unit I - Digital Logic Fundamentals